

The Query Cannot See the Question

A short convolution’s reach decides which part of a question conditions retrieval from a model’s memory

Maximiliano Speranza

Independent Researcher, Buenos Aires, Argentina

[ORCID 0009-0005-0413-8554](#)

September 2026

In one line. In linear-attention and state-space models there is a part of every question that the internal search **literally cannot see**, and what falls outside does not degrade gracefully: **it disappears exactly**. The model then answers confidently from the part it can still see.

1. What it is

A model with memory has to do two things that are usually measured together and are in fact separate. It has to *find* the right entry, and it has to *know* when the entry it is looking for is not there. The second one is what separates a memory from a confabulator.

What we found is that the second one can fail for a purely mechanical reason, upstream of everything the model learns. The query used to search the memory is not built from the whole question. It is built by a **short causal convolution** over the hidden states, and that convolution has a fixed reach of a few tokens.

If part of the question falls outside that reach, it does not influence the search a little. **It does not influence it at all.**

2. How it works

The default kernel size of that convolution is **4**. That means the query at a given layer is a function of the current token and **three predecessors**, and of nothing else.

That number is usually inherited rather than measured. Mamba-2 uses 4 by default. The reference implementation of Qwen3-Next exposes `linear_conv_kernel_dim`, documented as “kernel size of the convolution used in linear attention layers”, also defaulting to 4. A reach of three tokens is not an exotic configuration, it is the factory setting in deployed models.

The concrete failure it produces

Consider a question of the form “*what is the [relation] of [entity]?*”. Read from the end, the **entity** sits 1 token away and the **relation** 3. With a reach of 2, the relation falls **exactly one token outside**, deterministically, in 100% of queries.

The model therefore searches **using the entity alone**. And there the failure that most closely resembles a real hallucination appears: asked about a relation that was **never stated**, for an

entity that **was**, it does not abstain. It retrieves the neighbouring fact, the one it does have stored for that entity, and delivers it confidently.

It was not ignoring the relation. **It could not see it.**

3. How it was found

The intervention is simple enough to repeat on any architecture, and it requires no training:

Change one token of the question and see whether the search moves.

Pick a read position, replace **a single token** at distance d behind it, and measure how much the convolution output moved at that position.

Measurement 1 · the real reach is not the nominal one

Before measuring the effect one has to measure the window, because a weight can be zero, in which case the effective reach is smaller than the nominal one. In `mamba-130m-hf`:

layer	max weight	per tap, oldest to current		
0	0.000000	0.933774	1.396596	3.513000
1	0.000000	0.599272	0.928984	1.315802
12	0.000000	0.309363	0.901561	0.981588
23	0.000000	0.128891	0.611719	1.090700

The oldest tap is exactly zero in all 24 layers. Not small: zero, across all 1,536 entries of each layer. The effective reach of this checkpoint is **2 tokens**, not 3.

We report the measurement and not a cause. A weight that is exactly zero across 24×1536 entries does not come from a gradient, so a conversion artefact is plausible, but we did not verify that and we do not claim it. The practical consequence, if it generalises, is that **the nominal kernel is an upper bound on the reach and should be measured rather than assumed.**

Measurement 2 · the cut is exact, and the layer sees what the query does not

Here is the central result, and the part that surprises. With the same intervention we measure two things: how much the **query** moves, and how much the **layer output** moves.

distance	query	layer output
$d = 1$	1.100457	0.074570
$d = 2$	0.579417	0.057475
$d = 3$	0.000000	0.017182
$d = 4$	0.000000	0.024114
$d = 5$	0.000000	0.014883
$d = 6$	0.000000	0.012871
$d = 7$	0.000000	0.017005

Two things, and the second is what makes this non-obvious.

There is no decay. The query’s movement goes from a large value to 0.000000 in one step, exactly where the measured reach ends.

And the model does see that token. The layer output moves at *every* distance, because recurrence carries the information forward. That is:

The state sees the whole sequence; the query that reads it does not.

That is why the problem is invisible from the outside. The model *has* the information. What it cannot do is use it to decide what to look for.

4. What it is good for

A diagnosis that costs nothing

Changing one token and checking whether retrieval moves requires no training, no gradients and no labels. If it does not move, that token is **invisible** to the search, and any confidence the model expresses about it is **unearned**.

A fix that costs 0.12%

Widening the kernel of the query-forming convolution from 3 to 5 raises the reach from 2 to 4, and the missing part enters the window. Measured on a small model trained from scratch, three seeds per condition:

	kernel 3	kernel 5
correct abstention in the hard case	0.5850 – 0.7349	0.9931 – 1.0000
overall accuracy	0.91 – 0.93	0.988 – 0.993

With no overlap between conditions, against a trivial floor of 0.4065. And it costs **1,024 parameters out of 865,651**, that is **0.12%** of the model: two more taps, by 128 channels, by 4 blocks.

It is not capacity. It is access.

And it is not a corner case

Measured over **33,585 questions** from four public corpora (SQuAD, Natural Questions, TriviaQA and HotpotQA), between **90% and 100%** of real questions have their discriminating parts further apart than the measured reach. And it **gets worse with difficulty**: on multi-hop questions it reaches 1.0000.

The configuration our synthetic task was built to study is the ordinary case.

5. What this does NOT claim

The limit is part of the result, so it is worth stating explicitly.

In a deep model the window attenuates rather than blocks. The cut is exact at layer 0, but from layer 1 recurrence restores the signal attenuated. We measured the rate: 1.077 and

1.028 per token of distance in 24- and 48-layer models, with $r = 0.978$ in both. Three times the parameters and twice the depth leave the rate unchanged, which is what makes the attenuation a property of the architecture rather than of a particular checkpoint.

The behavioural effect is transient when there are layers to spare. Fine-tuning a 24-layer model on the same task, the difference exists and is large at step 100, and **disappears entirely by step 400**. This is a negative result, it was pre-registered, and we report it.

**Where there are no layers left to pay with, the window sets a ceiling.
Where there are, it sets a toll.**

The strong claim therefore holds for **memory consulted from an early layer**, and not as a behavioural prediction for a deep model trained to convergence.

6. Where it applies, by scale

	depth	what happens
small model	4 blocks, read at block 0	permanent ceiling
mamba-130m / 370m	24 and 48 layers	toll , the effect recovers
frontier models	tens of layers, hybrid	not measured

On frontier models the honest statement is this: **the mechanism is present** — kernel 4 is the default in Mamba-2 and in the linear-attention layers of Qwen3-Next — but **whether it produces failures in a large model trained to convergence we did not measure and do not claim**. Two reasons push against it: if 24 layers suffice to pay the toll, 60 are likely to suffice more; and frontier models are hybrid, where a full-attention layer does not have this limitation because its query sees the whole sequence.

Where it is reasonable to expect it to matter in a large system is when **memory is consulted from an early layer**, as in retrieval conditioned on early-layer states. There are no layers upstream to pay the toll with, and the ceiling returns.

The lesson that generalises. The query used to search a memory has to be formed where the whole question has already been seen. And it can be checked cheaply on any architecture, without training anything: change one part of the query and see whether retrieval moves. If it does not, that part is invisible.